

Final Report: Binary Analysis with AI PalmTree Reproduction, Open-Source Pipeline, and Ablation Study

Montassar Arfaoui
LMU Munich
Munich, Germany
Email: arfaoui.montassar@campus.lmu.d

1. Introduction

This report presents the final outcome of the **Binary Analysis with AI** practical course project. The focus of this project was the reproduction, modification, and evaluation of **PalmTree** [1], a pre-trained assembly language model designed for instruction embedding.

This report focuses on the conceptual contributions, implementation decisions, experimental setup, and interpretation of results. Detailed scripts, commands, and execution instructions are available in the accompanying repository and README file.

The original PalmTree is a pre-trained assembly language model designed to learn instruction embeddings for binary analysis tasks. It is built on top of a lightweight BERT-style transformer architecture and trained in a self-supervised manner on large collections of compiled binaries.

PalmTree improves over earlier approaches such as word2vec and Asm2Vec by explicitly modeling both syntactic and semantic relationships. It achieves this using three training objectives:

- 1) **Masked Language Modeling (MLM)**: Predict masked tokens within an instruction to learn syntax.
- 2) **Context Window Prediction (CWP)**: Predict whether two instructions appear close in a basic block, capturing control-flow structure.
- 3) **Def-Use Prediction (DUP)**: Predict whether one instruction defines a value that another instruction later uses, modeling data-flow dependencies.

Although PalmTree is open source, its preprocessing pipeline depends on Binary Ninja, a commercial reverse-engineering tool. This dependency restricts reproducibility and limits accessibility in academic environments.

2. Overview

The project was structured around two primary objectives.

2.1. Objective 1: Fully Open-Source PalmTree Pipeline

The first objective was to rebuild PalmTree’s preprocessing pipeline using only open-source tools. The required

components of the fully open-source preprocessing pipeline include binary disassembly, basic block construction, instruction tokenization, control-flow pair generation for CWP, and def-use pair generation for DUP.

Binary Ninja was replaced with **angr**, an open-source binary analysis framework. This ensures that the entire pipeline from raw binaries to training data is reproducible without proprietary software.

2.2. Objective 2: Ablation Study of Training Objectives

The second objective was to analyze how each training objective contributes to overall performance. The following variants were trained:

Model Variant	MLM	CWP	DUP
PalmTree-M	X	–	–
PalmTree-MD	X	–	X
PalmTree-MC	X	X	–
Full PalmTree	X	X	X

This design isolates the effect of CWP and DUP when combined with MLM.

3. Implementation/Design

3.1. Binary Disassembly and Pair Generation

The first step was implementing binary disassembly using **angr**. From raw x86-64 binaries:

- Control Flow Graphs (CFGs) were extracted.
- Data Flow Graphs (DFGs) were constructed.
- Instruction pairs were derived from CFG edges (for CWP).
- Def-use pairs were derived from DFG edges (for DUP).

Def-use relations connect instructions that define a value (e.g., writing to a register) to later instructions that use that value, capturing semantic dependencies beyond mere positional proximity.

The tokenization strategy followed PalmTree’s fine-grained approach, preserving symbolic structure rather than treating instructions as plain text.

```

call rax      add rsp 8
add rsp 8     ret
add rsp 8     ret
sub rsp 8     mov rax qword ptr [ rip + address ]
mov rax qword ptr [ rip + address ] test rax rax
test rax rax  je address
je address    call rax
call rax      add rsp 8
add rsp 8     ret
mov rax qword ptr [ rip + address ] test rax rax
test rax rax  je address
je address    call rax
call rax      add rsp 8
add rsp 8     ret
test rax rax  je address
je address    add rsp 8
add rsp 8     ret
je address    call rax
call rax      add rsp 8
add rsp 8     ret
push qword ptr [ rip + address ] jmp qword ptr [ rip + address ]
xor ebp ebp  mov r9 rdx
mov r9 rdx    pop rsi
pop rsi mov rdx rax
mov rdx rax  and rsp address
and rsp address push rax
push rax      push rsp
push rsp      mov r8 symbol
mov r8 symbol mov rcx symbol
mov rcx symbol mov rdi symbol
mov rdi symbol call qword ptr [ rip + address ]
mov r9 rdx    pop rsi
pop rsi mov rdx rax
mov rdx rax  and rsp address
and rsp address push rax
push rax      push rsp
push rsp      mov r8 symbol
mov r8 symbol mov rcx symbol
mov rcx symbol mov rdi symbol
mov rdi symbol call qword ptr [ rip + address ]
mov rdx rax  and rsp address
and rsp address push rax
push rax      push rsp
push rsp      mov r8 symbol
mov r8 symbol mov rcx symbol
mov rcx symbol mov rdi symbol
mov rdi symbol call qword ptr [ rip + address ]
and rsp address push rax
push rax      push rsp
push rsp      mov r8 symbol
mov r8 symbol mov rcx symbol

```

Figure 1. Example part of generated DFG (def–use) pairs from the text file.

3.2. Environment and Data Separation

To ensure reproducibility and modularity, the project was divided into two conda environments:

- **angr environment:** input generation and graph extraction.
- **palmtree-train environment:** model training and evaluation (PyTorch).

The dataset was strictly separated:

- 5 projects for training
- 2 projects for evaluation

This prevents data leakage and ensures meaningful retrieval evaluation.

3.3. Tensor Preprocessing Optimization

Initial experiments revealed preprocessing bottlenecks during training: tensors were being constructed dynamically inside the training loop, which led to increased CPU usage, longer training times, and occasional out-of-memory errors.

To resolve this, preprocessing was moved offline, fixed-length tensors were generated once, and the resulting tensors were persisted to disk and reused across runs. This change significantly improved efficiency and training stability.

To resolve this:

- Preprocessing was moved offline.
- Fixed-length tensors were generated once.
- Tensors were persisted to disk and reused.

This significantly improved efficiency and training stability.

4. Evaluation

4.1. Research Question

The ablation study addresses the following research question:

RQ: How does each training objective (MLM, CWP, DUP) contribute to retrieval performance, and which objective provides the strongest additional signal when combined with MLM?

4.2. Methodology

Evaluation followed a retrieval-based design in which anchors were functions taken from a given project, positives were the same functions compiled under different compiler settings, and negatives were different functions. Approximately 28,000 anchor functions were evaluated. Performance was measured using Recall@1, Recall@10, Mean Reciprocal Rank (MRR), and NDCG. Recall@1 reports the fraction of anchors for which the correct match is ranked first, while Recall@10 reports the fraction for which a correct match appears anywhere in the top 10 most similar candidates. MRR summarizes ranking quality by giving higher credit when the first correct match appears near the top of the list (and lower credit as it appears later). NDCG additionally evaluates the quality of the overall ranked list by rewarding correct results that appear earlier and applying a position-based discount, while also normalizing scores to make them comparable across queries. These metrics were chosen because they jointly capture both strict top-1 correctness (Recall@1), practical “top-k” usability in retrieval scenarios (Recall@10), sensitivity to how early the first correct match appears (MRR), and the overall ranking quality beyond a single cutoff (NDCG).

4.3. Results and Analysis

4.3.1. Open-Source Pipeline Performance (Objective 1).

The results show strong and stable retrieval performance (see Fig.2), comparable embedding quality, and reduced computational overhead due to offline preprocessing. Overall, this confirms that a fully open-source pipeline using angr is both feasible and efficient.

